Institut National des Postes et Télécommunications (INPT)

3rd year – Innovation & AMOA

AI-powered Knowledge Graph Analytics Platform

Big Data, LLM extraction, Neo4j and GraphRAG:
a comparative evaluation

Project report – Big Data Analytics

Authors

Azzeddine Abouaam

Zakaria Ezzaami

Academic year 2026–2027

Abstract

This report presents the design, implementation and evaluation of an AI-powered knowledge graph analytics platform. Information is collected from heterogeneous sources (websites, a REST API, PDF and Word documents, e-mails, plain-text files and a relational database), stored in HDFS, and converted into entities and relationships by a large language model (LLM). A rule-based entity-resolution step and a Spark ETL job validate, deduplicate and load the result into a Neo4j knowledge graph. On top of this single graph we compare two analytical approaches: *structured graph analytics* (Cypher queries and Neo4j Graph Data Science algorithms) and *GraphRAG* (graph retrieval combined with LLM generation). The whole system runs as a Docker Compose proof of concept with a local, CPU-only model (`qwen2.5:3b`).

On a hand-annotated benchmark of 12 questions, the LLM extracts entities with an F1 of 0.86 and relations with an F1 of 0.71 after schema validation. Entity resolution removes every duplicate node and doubles Cypher accuracy on multi-hop questions (0.17 to 0.33). Cypher answers in about 10 ms with no inference cost and wins on aggregation; GraphRAG takes about 57 s and 1 200 tokens per question but wins on semantic questions and can answer from entity descriptions when an edge is missing. Every answer is backed by source citations, none of them invalid.

Contents

1 Introduction **3**
- 1.1 Context . . . 3
- 1.2 Objectives . . . 3
- 1.3 Research questions . . . 3
- 1.4 Report structure . . . 4

2 Architecture **5**
- 2.1 Overview . . . 5
- 2.2 Technology stack and deployment . . . 6
- 2.3 Graph schema . . . 6

3 Implementation **7**
- 3.1 Phase 1 – Collection and HDFS storage . . . 7
- 3.2 Phase 2 – LLM extraction and entity resolution . . . 8
- 3.3 Phase 3 – Spark ETL and Neo4j ingestion . . . 8
- 3.4 Phase 4 – Graph analytics . . . 9
- 3.5 Phase 5 – GraphRAG . . . 9
- 3.6 Phase 6 – Evaluation module . . . 9
- 3.7 Phase 7 – Integrated application . . . 9

4 Dataset and ground truth **10**
- 4.1 Corpus . . . 10
- 4.2 Ground truth . . . 10

5 Evaluation methodology **12**

6 Results **13**
- 6.1 **RQ1** – Extraction accuracy . . . 13
- 6.2 **RQ2** – Effect of entity resolution . . . 13
- 6.3 **RQ3** – Cypher versus GraphRAG . . . 13
- 6.4 **RQ4** – Latency and cost . . . 15
- 6.5 **RQ5** – Provenance . . . 15

7 Discussion **16**
- 7.1 Trade-offs . . . 16
- 7.2 Limitations . . . 16
- 7.3 Responsible use . . . 16

8 Conclusion and future work **18**

A Running the platform **19**

B Project structure **20**

Chapter 1

Introduction

1.1 Context

Organizations hold knowledge scattered across web pages, reports, e-mails, office documents and databases. The facts that matter most – who leads which company, who invested in whom, which supplier serves which customer – are rarely stored in one place, and answering a question often requires connecting facts found in different documents (*multi-hop* reasoning). Knowledge graphs model this naturally: entities become nodes and facts become typed edges. Two recent developments make building and querying such graphs much cheaper:

- **LLM-based information extraction**, which turns unstructured text into entities and relations without task-specific training;
- **GraphRAG**, which retrieves a relevant subgraph and lets an LLM answer natural-language questions over it, instead of requiring the user to write a query language.

1.2 Objectives

The goal of the project is to collect information from heterogeneous sources, extract entities and relationships, build a knowledge graph, and **compare traditional graph analytics with LLM-based graph retrieval and reasoning**. The key design principle is to maintain *one knowledge graph supporting two analytical approaches*, so that the comparison is fair: both approaches use exactly the same underlying data.

1.3 Research questions

RQ1 How accurately can an LLM extract entities and relationships from heterogeneous documents?

RQ2 How does entity resolution affect the quality of the resulting knowledge graph?

RQ3 How does GraphRAG perform compared with direct Cypher queries for multi-hop and semantic questions?

RQ4 What trade-offs exist between answer quality, completeness, latency and inference cost?

RQ5 How can source provenance improve the reliability and explainability of generated answers?

1.4 Report structure

Chapter 2 describes the architecture. Chapter 3 details the implementation, phase by phase. Chapter 4 presents the dataset and ground truth, Chapter 5 the evaluation method and Chapter 6 the results. Chapter 7 discusses trade-offs, limitations and responsible use, and Chapter 8 concludes.

Chapter 2

Architecture

2.1 Overview

Figure 2.1 shows the data flow. Sources are collected by a crawler and landed in HDFS. An LLM extracts entities and relations, which are resolved and then validated, deduplicated and transformed by Spark before being loaded into Neo4j. The graph then feeds two branches – graph analytics and GraphRAG – whose outputs are compared by the evaluation module.

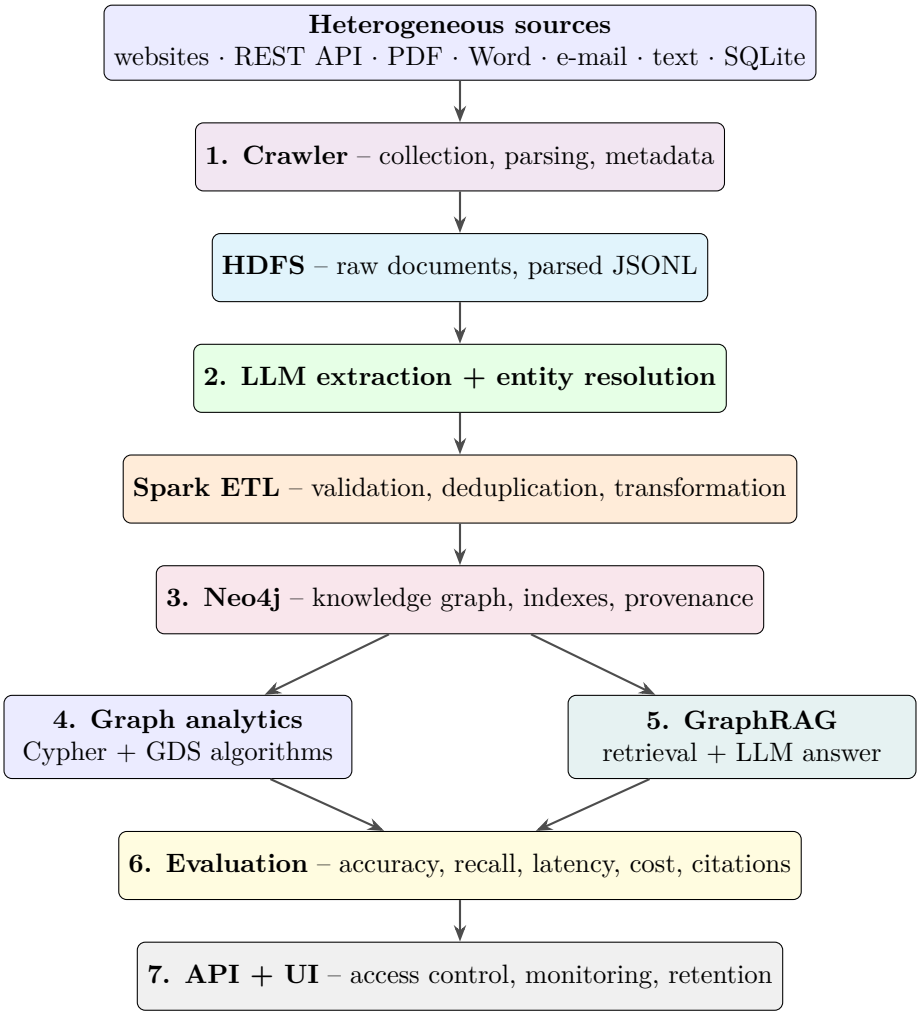


Figure 2.1: Platform architecture: one knowledge graph, two analytical approaches.

2.2 Technology stack and deployment

The proof of concept is a single Docker Compose project. Table 2.1 lists the services.

Service	Image	Port	Role
`namenode`	`apache/hadoop:3.4.1`	9870	HDFS NameNode (WebHDFS API, web UI)
`datanode`	`apache/hadoop:3.4.1`	–	HDFS DataNode (replication 1)
`neo4j`	`neo4j:5-community`	7474, 7687	Graph database + Graph Data Science plugin
`ollama`	`ollama/ollama`	–	Local LLM server, OpenAI-compatible API
`website`	`python:3.12-slim`	–	Demo website and JSON API crawled by the platform
`app`	custom (Python 3.12, Java 21)	8080	Crawler, extraction, Spark ETL, analytics, GraphRAG, evaluation, API/UI

Table 2.1: Docker Compose services.

The application image bundles PySpark 4.0 with a Java 21 runtime so that the Spark job runs next to the Python services. The LLM is accessed through an OpenAI-compatible endpoint: by default a local Ollama server running `qwen2.5:3b`, but any compatible provider can be used by setting `LLM_BASE_URL` and `LLM_MODEL`.

2.3 Graph schema

A shared, explicit schema constrains both the LLM and the validation step (Table 2.2). Every entity carries the label `Entity` plus its type label, a `description` and the list of source documents. Every relation carries `sources`, `evidence` (quotes from the text), `confidence` and `mentions`. Documents are nodes too, linked by `MENTIONED_IN`.

Relation	Source type	Target type
`FOUNDED`, `LEADS`, `WORKS_FOR`, `BOARD_MEMBER_OF`	Person	Organization
`INVESTED_IN`, `ACQUIRED`, `SUPPLIES`	Organization	Organization
`PARTNERS_WITH` (symmetric)	Organization	Organization
`DEVELOPS`	Organization	Product
`LOCATED_IN`	Organization	Location

Table 2.2: Relation types and their type signatures (entity types: Person, Organization, Product, Location).

Chapter 3

Implementation

The implementation follows the seven phases of the project roadmap (Table 3.1).

Phase	Scope	Deliverable	Module
1	Sample documents, crawler, HDFS	Working ingestion pipeline	`crawler.py`
2	LLM extraction, entity resolution	Structured entities and relationships	`extract.py`, `resolve.py`
3	Spark transformations, Neo4j ingestion	Populated knowledge graph	`spark_etl.py`
4	Cypher queries, graph algorithms	Graph analytics module	`analytics.py`
5	GraphRAG retrieval and generation	Natural-language graph assistant	`graphrag.py`
6	Ground truth, comparative tests	Benchmark report	`evaluate.py`
7	API, interface, monitoring, access control	Integrated application	`api.py`

Table 3.1: Roadmap phases and corresponding modules.

3.1 Phase 1 – Collection and HDFS storage

The crawler handles every source family of the architecture:

- **Websites**: breadth-first crawl from seed URLs, restricted to the seeds' hosts, using the standard library HTML parser to extract the title, visible text and links;
- **APIs**: each item of a JSON array becomes one document;
- **Files**: PDF (`pypdf`), Word (`python-docx`), e-mail (`email` standard library, headers kept as metadata) and plain text;
- **Databases**: each row of each SQLite table becomes a *column: value* document.

Each document receives a stable identifier (e.g. `pdf:northwind_report.pdf`), a SHA-256 hash used for deduplication, and a collection timestamp. Original bytes are written to `/raw/<kind>/` and parsed documents to `/parsed/docs.jsonl` through the WebHDFS REST API. The run collected 12 documents from 7 source kinds.

3.2 Phase 2 – LLM extraction and entity resolution

Extraction. One LLM call per document, at temperature 0 and in JSON mode. The prompt lists the allowed entity and relation types with their direction, requires an *evidence* quote and a *confidence* for every relation, and gives one worked example. A first version of the prompt without the example produced many reversed relations ("Helios Robotics FOUNDED Amara Okafor") and no locations; adding the one-shot example fixed most of them.

Entity resolution. Surface forms are clustered with a union-find structure over cleaned names (honorifics such as *Dr.* and legal suffixes such as *SA*, *LLP* or *S.p.A.* removed). Two names are merged when:

1. their normalized keys are identical (*Helios Robotics SA* = *Helios Robotics*);
2. one name's tokens are contained in exactly one other name (*Helios* → *Helios Robotics*); ambiguous containment is not merged;
3. they match by initials (*A. Okafor* → *Amara Okafor*);
4. their spelling is nearly identical (`difflib` ratio ≥ 0.92).

The canonical name is the most frequent cleaned form. On the corpus, 28 surface names were reduced to 20 entities. Both a resolved and an unresolved version are written to HDFS so that the effect of resolution can be measured.

3.3 Phase 3 – Spark ETL and Neo4j ingestion

The Spark job reads the extracted JSONL from HDFS and performs three steps:

- **Validation**: relation types are normalized (*CEO* → `LEADS`); relations that violate the type signature are *repaired* by swapping source and target when the reversed direction fits, and dropped otherwise; self-loops are removed;
- **Deduplication**: one node per name (majority type vote), one edge per (source, type, target), with the union of source documents, all evidence quotes, the maximum confidence and the mention count;
- **Transformation and loading**: the curated layer is written as Parquet to HDFS, then loaded into Neo4j with batched `UNWIND ... MERGE` statements executed in parallel from Spark partitions.

Listing 3.1: Schema validation with direction repair (shared by Spark and the evaluator).

```
want_s, want_t = SIGNATURES[t]
def fits(a, b): # unknown endpoint types (None) are accepted
    return a in (None, want_s) and b in (None, want_t)
if not fits(st, dt):
    if not fits(dt, st):
        return None # cannot fit the schema: drop
    src, dst = dst, src # reversed by the LLM: repair
```

Neo4j constraints guarantee unique entity names and document ids, and a full-text index covers entity names and descriptions for retrieval. The run produced 20 entity nodes and 24 deduplicated edges from 29 extracted relations.

3.4 Phase 4 – Graph analytics

The analytics module exposes direct Cypher queries and Graph Data Science algorithms on an in-memory projection of the entity graph:

- **Overview** and **degree centrality**;
- **PageRank** – most influential entities (Helios Robotics, Verdant Grid, Northwind Capital);
- **Betweenness** – brokers connecting otherwise distant parts of the graph (Northwind Capital, the investor, ranks first);
- **Louvain communities** – three clusters were found: the Verdant Grid / Corvid Analytics ecosystem, the Helios Robotics / Northwind cluster, and the Lumen Batteries / Quanta Bio cluster;
- **Shortest paths** with provenance, e.g. Mei Tanaka → Corvid Analytics → Verdant Grid ← Northwind Capital, each hop listing its source documents.

3.5 Phase 5 – GraphRAG

GraphRAG answers a natural-language question in three steps:

1. **Seed retrieval**: content words of the question (stop words removed, fuzzy matching for longer words) query the full-text index; seeds scoring below half of the best score are discarded, because a small model loses track of the question when the context covers the whole graph;
2. **Subgraph expansion**: all relations within two hops of the seeds, one-hop facts first, with their sources, evidence and confidence, plus the descriptions of the entities involved;
3. **Generation**: the LLM answers using only these facts, cites source ids in brackets after each claim, and presents facts as claims from sources rather than verified truth. Cited ids are checked against the retrieved context.

3.6 Phase 6 – Evaluation module

`evaluate.py` computes extraction metrics against the gold annotations, measures the resolution ablation, then loads the unresolved and the resolved graph in turn and runs every benchmark question through both approaches. Results are written to `reports/benchmark.json` and `reports/benchmark.md`.

3.7 Phase 7 – Integrated application

A FastAPI service exposes the platform through a REST API and a minimal web interface:

- `POST /ask` (GraphRAG), `GET /analytics/{name}`, `GET /path`, `GET /entity`;
- **access control**: API keys mapped to *reader* and *admin* roles (401 without a key, 403 for an insufficient role), every request audit-logged, question length validated;
- **monitoring**: `/health` probes Neo4j and HDFS; `/metrics` exposes request counts, latencies and LLM token usage in Prometheus format;
- **data retention**: `POST /admin/retention` deletes documents older than a configurable number of days, then every fact and entity no longer backed by any source.

Chapter 4

Dataset and ground truth

4.1 Corpus

To evaluate the platform without processing data about real people, we built an **entirely fictional** corpus describing a small European technology ecosystem (Table 4.1). It was designed to stress the two difficult points of the pipeline:

- **facts spread across sources**: answering most multi-hop questions requires combining at least two documents of different formats;
- **inconsistent naming**: the same entity appears as *Dr. Amara Okafor*, *Okafor* or *A. Okafor*, and *Northwind Capital Partners LLP*, *Northwind Capital* or *Northwind*.

Source type	Document(s)	Content
Website (crawled)	`index.html`, `helios.html`, `verdant.html`	Company profiles linked from an index page
REST API	`news.json` (3 items)	Investment and partnership news
PDF	`northwind_report.pdf`	Investor annual portfolio report
Word	`lumen_brief.docx`	Battery manufacturer company brief
E-mail	`corvid_partnership.eml`	Partnership announcement from a founder
Text file	`basel_notes.txt`	Field notes
Database	`registry.sqlite` (2 rows)	Company registry table

Table 4.1: Fictional sample corpus: 12 documents from 7 source types.

4.2 Ground truth

The gold file contains 24 entities with their aliases, 28 relations and 12 questions. Each question has its type, gold answer (with accepted variants), supporting source documents, a reference Cypher query (the *graph analytics* approach) and, for multi-hop questions, the gold path. Table 4.2 lists the questions.

Listing 4.1: Reference Cypher query for the multi-hop question Q7.

```
MATCH (p:Person)-[:BOARD_MEMBER_OF]->(h:Entity),
      (p)-[:WORKS_FOR]->(i:Entity)-[:INVESTED_IN]->(h)
WHERE toLower(h.name) CONTAINS 'helios'
RETURN DISTINCT p.name
```

Id	Type	Question	Answer
Q1	single-hop	Who founded Helios Robotics?	Amara Okafor
Q2	single-hop	In which city is Verdant Grid headquartered?	Oslo
Q3	single-hop	Which companies has Northwind Capital invested in?	Helios, Verdant, Lumen
Q4	aggregation	How many companies has Northwind Capital invested in?	3
Q5	multi-hop	In which city is the battery supplier of Helios Robotics located?	Turin
Q6	multi-hop	Who leads the company that partners with Corvid Analytics?	Lars Henriksen
Q7	multi-hop	Which board member of Helios Robotics works for one of its investors?	Daniel Weiss
Q8	multi-hop	Who founded the company that partners with the developer of GridMind?	Mei Tanaka
Q9	multi-hop	Which company did the developer of GridMind acquire, and where is it based?	Kestrel, Bergen
Q10	multi-hop	Which company supplies two different companies Northwind invested in?	Lumen Batteries
Q11	semantic	Which company works on energy storage technology?	Lumen Batteries
Q12	semantic	Which organization automates biotech laboratories with robots?	Quanta Bio

Table 4.2: Benchmark questions.

Chapter 5

Evaluation methodology

- **Extraction (RQ1)**: precision, recall and F1 of entities and relations against the gold annotations. Extracted names are mapped to gold entities through the alias table (lenient matching), so that this metric isolates the LLM's extraction quality from naming. Relations are measured twice: as returned by the LLM and after Spark schema validation.
- **Entity resolution (RQ2)**: on the unresolved and resolved graphs, number of nodes, duplicate nodes (several nodes for one gold entity), fragmentation (nodes per gold entity) and *walkable multi-hop paths*: a gold path counts only if its hops are connected through *identical* nodes, i.e. if a graph traversal could actually follow it. QA accuracy is also measured on both graphs.
- **Question answering (RQ3)**: for each question, a gold item is found if one of its accepted forms appears (whole word, case-insensitive) in the answer. *Recall* is the share of gold items found; a question is *correct* when recall is 1. Cypher results and GraphRAG answers are scored the same way.
- **Cost (RQ4)**: end-to-end latency per question and LLM tokens, at extraction and query time.
- **Provenance (RQ5)**: share of answers with citations, invalid citations (ids not present in the retrieved context) and citation precision (share of cited documents that are gold supporting documents).

All experiments ran on a laptop CPU (8 cores, no GPU) with `qwen2.5:3b` at temperature 0.

Chapter 6

Results

6.1 RQ1 – Extraction accuracy

Level	Precision	Recall	F1	TP	Predicted	Gold
Entities	1.000	0.750	0.857	18	18	24
Relations (LLM output)	0.692	0.643	0.667	18	26	28
Relations (after validation)	0.783	0.643	0.706	18	23	28

Table 6.1: Extraction quality of `qwen2.5:3b`.

Entity extraction is precise (no false entity) but misses a quarter of the gold entities, mostly cities mentioned indirectly ("an Oslo-based company") and secondary mentions. Relation extraction is the weaker step: the model confuses relation types (Daniel Weiss `LEADS` instead of `WORKS_FOR` Northwind) and sometimes reverses directions. Spark validation drops or repairs schema-violating relations, raising relation precision from 0.69 to 0.78 at no cost in recall.

6.2 RQ2 – Effect of entity resolution

Graph	Nodes	Gold-matched	Gold found	Duplicates	Fragm.	Walkable
Unresolved	28	26	20	6	1.30	1/6
Resolved	20	20	20	0	1.00	2/6

Table 6.2: Graph quality with and without entity resolution.

Resolution removes all six duplicate nodes without merging distinct entities, and doubles the number of gold multi-hop paths that a traversal can follow. Downstream, Cypher accuracy on multi-hop questions doubles from 0.17 to 0.33 (Table 6.3): Q8, for instance, fails on the unresolved graph because *Corvid Analytics* and *Corvid Analytics Inc.* are two different nodes. The remaining unwalkable paths are caused by relations that were never extracted, which resolution cannot fix.

6.3 RQ3 – Cypher versus GraphRAG

Each approach has clear strengths (Figure 6.1):

- **Cypher is exact when the graph is right.** It answers the aggregation question perfectly, while GraphRAG over-counts (it listed four investments instead of three). But it is

Graph	Question type	n	Cypher		GraphRAG	
			accuracy	recall	accuracy	recall
Unresolved	single-hop	3	0.67	0.67	1.00	1.00
	aggregation	1	1.00	1.00	0.00	0.00
	multi-hop	6	0.17	0.25	0.50	0.58
	semantic	2	1.00	1.00	0.50	0.50
	all	12	**0.50**	0.54	**0.58**	0.63
Resolved	single-hop	3	0.67	0.67	0.67	0.67
	aggregation	1	1.00	1.00	0.00	0.00
	multi-hop	6	0.33	0.42	0.50	0.58
	semantic	2	0.50	0.50	1.00	1.00
	all	12	**0.50**	0.54	**0.58**	0.63

Table 6.3: Question-answering accuracy and recall by question type.

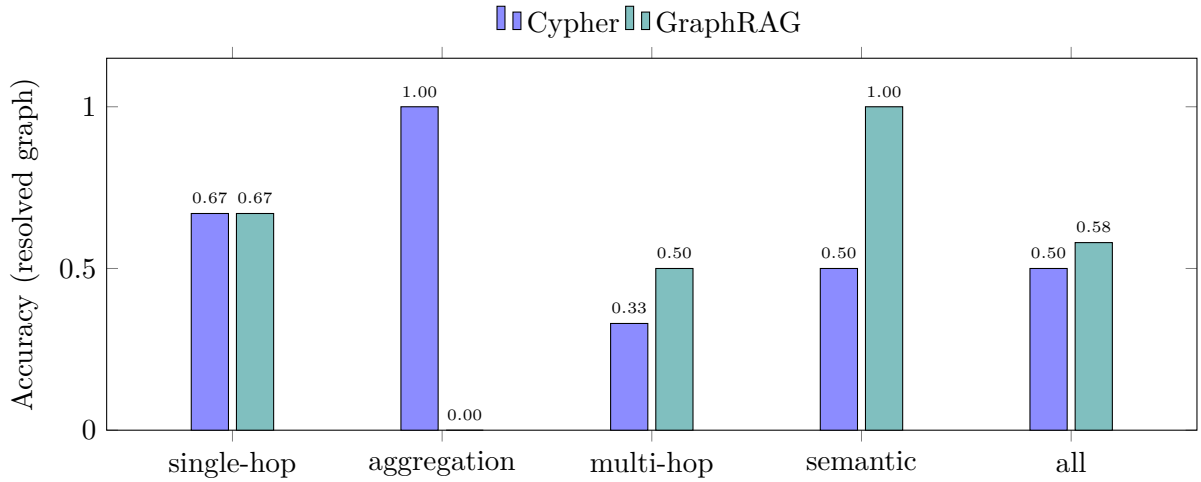


Figure 6.1: Accuracy of both approaches per question type on the resolved graph (automatic, lenient scoring).

brittle: a single missing or mistyped edge on the path returns nothing (Q5, Q6, Q7).

- **GraphRAG is more robust to graph gaps.** Because its context includes entity descriptions, it can answer from text even when an edge is missing: Q5 succeeds although the needed edge path is incomplete, and both semantic questions are answered, whereas keyword filters in Cypher depend on the exact wording of the descriptions.
- **Small-model reasoning is the main limit of GraphRAG.** On Q6, the retrieved context contained the answer (Lars Henriksen is described as CEO of Verdant Grid), yet the 3B model chained the wrong hops.

Scoring caveat. The automatic scoring is mention-based and therefore lenient for free-text answers. Reading the answers on the resolved graph, GraphRAG's answers to Q10 and Q12 mention the gold entity but designate a different one as the answer. With strict manual judgment, GraphRAG accuracy on the resolved graph is **5/12 (0.42)**, below Cypher's 6/12 (0.50). Cypher results are lists of values and are not affected.

	Cypher	**GraphRAG**
Mean latency per question	≈ 0.01 s	57.2 s
LLM tokens per question	0	1 178
Query authoring	expert writes Cypher	natural language
Shared one-off cost – extraction: 11 768 tokens, 517 s of LLM time (43 s per document)		

Table 6.4: Latency and inference cost (resolved graph, CPU only).

6.4 RQ4 – Latency and cost

Cypher is about four orders of magnitude faster and free at query time, but requires an expert who knows the schema. GraphRAG trades latency and tokens for accessibility. Both pay the same up-front extraction cost, which dominates the total for a small number of questions. On a GPU or a hosted model, GraphRAG latency would drop to a few seconds, but its per-question token cost remains.

6.5 RQ5 – Provenance

All 24 GraphRAG answers (12 per graph) contained citations, and none cited a document absent from the retrieved context. Citation precision against gold supporting documents was 0.79 on the unresolved graph and 0.54 on the resolved graph: once entities are merged, each node gathers sources from many documents and the model tends to cite all of them rather than only the supporting ones. Provenance makes errors *verifiable*: in the Q9 answer, the model claims Verdant Grid is based in Basel and cites the registry row about Quanta Bio – a reader following the citation immediately sees the mistake. Every edge also keeps its evidence quote, which the web interface displays next to the answer.

Chapter 7

Discussion

7.1 Trade-offs

- **Quality of the graph bounds both approaches.** Questions Q2 and Q7 fail for both approaches because the needed facts were not extracted. Improving extraction (larger model, chunking, multiple passes) helps more than improving either query approach.
- **Cypher for precise, repeated, aggregate questions**; **GraphRAG for exploratory and semantic questions** asked by non-experts. A practical system can combine them: GraphRAG for exploration, with Cypher (possibly generated by the LLM and validated) for counts and exact lists.
- **Entity resolution is cheap and high-value.** A few dozen lines of rules removed all duplicates and doubled walkable multi-hop paths, without any LLM cost.
- **Schema validation is a cheap safety net** for small models: repairing directions raised relation precision by 9 points.

7.2 Limitations

- The corpus is small (12 documents, 28 relations, 12 questions); results show tendencies, not statistically robust differences.
- Only one, small model was evaluated, because of hardware limits (CPU, about 2 GB of free memory).
- Answer scoring is automatic and lenient for free text (see the scoring caveat above).
- Spark runs in local mode inside the application container, with a single HDFS DataNode; the code is written for a cluster (`SPARK_MASTER`) but was not benchmarked on one.
- GraphRAG seeds rely on a full-text index; vector embeddings would improve fuzzy semantic matching.

7.3 Responsible use

When a platform of this kind analyzes networks of identifiable people, inferred associations can harm individuals. The design therefore includes:

- a fictional dataset for development and evaluation;
- provenance on every node and edge (source documents, evidence quotes, confidence);

- a GraphRAG prompt and a user-interface notice presenting answers as claims from cited sources, not verified facts;
- role-based API keys and audit logging of every request;
- a retention endpoint that removes old documents and every fact that depended only on them.

Any deployment on real data must restrict collection to lawful, authorized sources.

Chapter 8

Conclusion and future work

We built an end-to-end proof of concept of an AI-powered knowledge graph analytics platform: heterogeneous sources are crawled into HDFS, turned into a knowledge graph by an LLM, entity resolution and Spark, and analyzed in Neo4j by two approaches that share the same graph. The evaluation answers the five research questions: a small local LLM extracts entities reliably but relations only moderately well; entity resolution and schema validation are cheap steps with a large effect; Cypher is fast, exact and best for aggregation, while GraphRAG is slower and costlier but handles semantic questions and missing edges better; and source provenance makes every generated answer verifiable.

Future work includes:

- evaluating larger models (local 7B–14B or hosted) and measuring the quality/cost curve;
- a hybrid assistant that generates and validates Cypher for aggregate questions and uses GraphRAG otherwise;
- vector embeddings for retrieval and chunking for long documents;
- a larger corpus and an LLM-as-judge or human scoring of free-text answers;
- a distributed Spark cluster, incremental graph updates, and production authentication (OIDC).

Appendix A

Running the platform

```
docker compose up -d --build # HDFS, Neo4j+GDS, Ollama, website, app
docker compose exec ollama ollama pull qwen2.5:3b # once (~2 GB)
docker compose exec app python pipeline.py # crawl, extract, resolve, etl, analytics, evaluate
```

- Web interface and API: http://localhost:8080 (key `reader-demo-key`)
- Neo4j Browser: http://localhost:7474
- HDFS NameNode: http://localhost:9870
- Benchmark report: `reports/benchmark.md`

Listing A.1: API examples.

```
K='X-API-Key: reader-demo-key'
curl -H "$K" -H 'Content-Type: application/json' \
    -d '{"question":"Who founded Helios Robotics?"}' localhost:8080/ask
curl -H "$K" localhost:8080/analytics/pagerank
curl -H "$K" 'localhost:8080/path?a=Mei%20Tanaka&b=Northwind'
```

Appendix B

Project structure

```
docker-compose.yml hadoop.env README.md
app/ crawler.py extract.py resolve.py spark_etl.py analytics.py
     graphrag.py evaluate.py api.py pipeline.py common.py
     make_samples.py test_resolve.py static/index.html Dockerfile
data/ web/ files/ db/ gold/gold.json
reports/ benchmark.md benchmark.json
report/ rapport.tex
```